Tuning and Benchmarking of RocksDB

Meet Vaghasiya
MTech (ICT)
Dhirubhai Ambani University
Gandhinagar, Gujarat, India
202511020@dau.ac.in

Prof. P M Jat
Guide
Dhirubhai Ambani University
Gandhinagar, Gujarat, India
pm_jat@daiict.ac.in

***Abstract*—RocksDB is a high performance key-value store widely used in production systems. But achieving better performance requires some careful configuration settings of its various parameter tuning. This study presents an experimental study into how individual parameter modifications affect three important performance metrics: throughput, read latency (P99), and space amplification. Through systematic benchmarking using both db_bench and YCSB ons multiple parameter categories including compression algorithms, memtable size, background job parallelism, block cache configuration, I/O strategies, and compaction styles. We identify specific trade-offs and provide actionable tuning paths. Our findings demonstrate that LZ4 compression offers the best balance between disk usage and read performance, that 256-512 MB memtable sizes minimize compaction overhead while increase in space amplification, and that increasing background job threads provides near linear throughput improvements up to 16 threads. We also characterize how different workload patterns respond to parameter changes and provide practical decision frameworks for applications. The results emphasize that default configurations are not always optimal. instead workload characteristics must drive parameter choices.**

I Introduction

Modern data intensive applications from databases to realtime analytics systems depend on embedded key-value database to achieve high throughput in write workloads while maintaining reasonable read latencies. RocksDB as an LSM tree based key-value store developed by Facebook, has become a popular choice in production deployments including MyRocks, TiKV, CockroachDB, and Flink state backends. Despite its widespread adoption, practitioners often struggle to find maximum performance from RocksDB because the system exposes various of tunable parameters and each with subtle interactions and workload dependent trade-offs.

The issue is imcremented by the fundamental trade-offs frmo LSM tree design. The write amplification, read amplification, and space amplification metrics often conflict. It cannot be minimized all simultaneously. A configuration that reduces write amplification through small flushing may increase read latency by deepening the LSM tree. Same way very active compaction to flatten the tree consumes I/O bandwidth and can cause write stalls. These trade-offs are not just theoretical. They directly impact end user experience through throughput degradation, latency spikes, and runaway disk usage.

Current RocksDB documentation and prior research focus on either high level architecture and compaction strategy selection, or on isolated parameter studies. What is missing is a comprehensive empirical study that simultaneously explores how six categories of parameters like compression, memtable size, background thread parallelism, caching, I/O modes, and compaction techniques affect measurable performance outcomes across diverse workload patterns.

A. Motivation and Gaps

While previous work has examined different aspects of LSM tree optimization (e.g., compaction technique selection), the interaction effects between parameters and their behavior under realistic parameter configurations are not discussed well. Practitioners must currently rely on default settings or trial and error for tuning in production environments. This study aims to bridge that gap by providing empirical evidence for parameter choices and actionable guidance based on workload characteristics.

B. Contributions of this Study

This work makes the following contributions:

- **Systematic Parameter Study**: A controlled empirical evaluation of six major parameter categories using consistent benchmarking methodologies across both synthetic (db_bench) and realistic (YCSB) workloads, with clear documentation of each parameter's isolated impact.
- **Trade-off analysis**: Explicit measurement of how parameter changes affect the three amplification metrics and latency characteristics, enabling practitioners to understand the cost of optimization choices in other dimensions.
- **Workload-Specific Guidance**: Classification of which parameter choices are optimal for different access patterns (read-heavy, write-heavy, mixed, read-modify-write) and dataset sizes.
- **Practical Decision Framework**: A set of decision trees and configuration templates that practitioners can use to derive RocksDB configurations from workload and hardware characteristics without exhaustive benchmarking.

C. Scope of study

This paper is organized as follows. Section II provides background on LSM trees and RocksDB architecture sufficient to understand parameter impacts. Section III describes our experimental methodology, including benchmarking tools, parameter ranges, and measurement procedures. Sections IV

through VIII present findings for each parameter category, including quantitative results, trade-off analysis, and each category guidance. Finally, Section IX addresses study of limitations and future research directions.

II Background: RocksDB and LSM Trees

A. LSM Tree Fundamentals

The Log Structured Merge (LSM) tree is an append only data structure designed for optimized for write throughput at the cost of read latency. All writes in DB first go to an in-memory component called MemTable. It is implemented as a sorted skiplist or hash skiplist, where they are immediately visible to read query. When the MemTable reaches a size threshold, it is flushed to disk as an immutable Sorted String Table (SSTable) file at Level 0.

The main idea of LSM trees is that this write path involves only sequential disk I/O. The MemTable is written sequentially in batch in disk as an SSTable rather than the random I/O required by traditional B-trees. This sequential access pattern is very fast on both disks and SSDs, makes the high write throughput that makes LSM trees attractive for writeheavy workloads.

The downside appears in compaction. Without periodic merging and rewriting of SSTables in different levels, the number of files in L0 grows without bound, and reads must check more and more files to find the latest version of a key. RocksDB solves this with background compaction, it merges overlapping files from two adjacent levels, removes stale data versions, applies tombstones from deleted data, and writes the result as fewer, larger files at the next level. This compaction is necessary but costly. It reads all compacted files data, transforms it, and rewrites it, making write amplification.

B. The Three Amplification Metrics

RocksDB performance is characterized by three amplification metrics:

- **Write Amplification (WA)**: The ratio of total bytes write in disk to the logical bytes originally written by the application. High WA degrades write throughput, increases latency variability, and wears out SSDs early.
- **Read Amplification (RA)**: The number of files or levels a read operation must visit to find a key. High RA increases read tail latency (P99, P99.9) because reads may visit L0 files, intermediate levels, and eventually lower levels before finding the key or confirming it does not exist.
- **Space Amplification (SA)**: The ratio of physical disk used to the logical size of the database. High SA wastes storage resources and increases cloud infrastructure costs. SA grows when old data is kept due to lazy compaction or when the LSM tree has overlapping levels.

These three metrics are in fundamental conflict. Strategies that reduce WA (e.g., lazy compaction, fewer small flushes) tend to make the LSM tree deepen and increase RA. Strategies that reduce RA (e.g., early compaction) increase WA. Strategies that minimize SA require either aggressive compaction (high WA) or a small tree (high RA). RocksDB parameter tuning is fundamentally about choosing acceptable trade-offs given specific workload requirements.

C. RocksDB Architecture Overview

RocksDB is built on the LSM tree with two main components in-memory component and a disk structure:

- **MemTable**: A sorted in-memory data structure receive all writes. Configurable size (write_buffer_size). When full, transitions to immutable state.
- **Immutable MemTable Queue**: Up to (max_write_buffer_number - 1) is number of MemTables waiting to be flushed.
- **Block Cache**: An in-memory cache of recently read data blocks. Reduces repeated disk reads for frequently accessed keys.
- **Disk Levels L0 through Ln**: SSTables organized hierarchically. L0 files are uncompacted. Higher levels are sorted and organized for binary search. The size of each level grows by max_bytes_for_level_multiplier, default 10 times each level.

Background threads execute two tasks: (1) *Flushing* MemTables to disk as L0 files, and (2) *Compacting* overlapping files from adjacent levels into fewer, larger files. The number of concurrent flush and compaction threads is controlled by max_background_jobs, an important parameter for parallelism.

III Experimental Methodology

A. Benchmarking Tools and Workloads

We conducted experiments using two standard benchmarking tools:

- **db_bench**: It is a synthetic workload generator provided with RocksDB. It is designed to isolate the performance of specific operations. We used fillrandom and readrandom patterns to evaluate both write and read performance.
- **YCSB (Yahoo Cloud Serving Benchmark)**: A standard benchmark for evaluating cloud data serving systems with many different workload patterns: Workload A (50% read, 50% update), Workload B (95% read, 5% update), Workload C (100% read), and Workload F (read-modify-write).

B. Experimental Design

We adopted one parameter change strategy at a time, where each parameter is varied independently while keeping others at base values. This isolates the effect of each parameter and makes results interpretable, though it does not capture all interaction effects (discussed in limitations).

1) Baseline Configuration

Our baseline configuration is RocksDB's default for most parameters, with the following exceptions chosen for realistic deployment:

```
block_cache_size: 1 GB
max_background_jobs: 4
write_buffer_size: 64 MB
compression: lz4
compaction_style: level
target_file_size_base: 64 MB
```

2) Parameter Ranges

Six parameter categories were evaluated:

- **Compression**: none, snappy, lz4, zstd (CPU/storage trade-offs)
- **Memtable size** (write_buffer_size): 64, 128, 256, 512 MB (flushing frequency trade-offs)
- **Background jobs** (max_background_jobs): 2, 4, 8, 16 (parallelism levels)
- **Block cache size**: 512 MB, 1 GB, 2 GB, 4 GB (memory availability trade-offs)
- **Compaction style**: level, universal, FIFO (structural trade-offs)

C. Measurement and Metrics

For each experiment, we recorded following things:

- **Throughput**: Operations per second (readrandom for db_bench, mixed operations for YCSB)
- **Read Latency (P99)**: 99th percentile read latency in microseconds; we use this instead of mean latency because tail latency is critical for user facing applications.
- **Database Size**: Physical disk usage of the database in MB, reflecting space amplification.
- **Compaction Metrics**: Number of flushes, flush time, number of compactions, compaction time (proxies for internal I/O overhead).

D. Experimental Environment

All experiments were conducted on Windows Subsystem for Linux 2 (WSL2) with the following characteristics:

- **Processor**: Intel(R) Core(TM) i5-10505 CPU @ 3.20GHz with 12 threads available
- **Storage**: WSL2 mounted NTFS filesystem (drvfs) on an external SSD
- **Memory**: 16 GB total RAM (7.6 GB dedicated), 256 GB SSD NVMe (244 GB dedicated)
- **RocksDB Version**: Latest stable release 11.6.0

1) Study Limitations

The experimental environment and methodology have important limitations:

1) **Storage Layer**: WSL2/drvfs is not representative of production Linux deployments. NTFS filesystem behavior differs from ext4 or XFS; results may not generalize to native Linux environments.
2) **Dataset Scale**: Experiments use 8–100 million records with 1 KB values. Real RocksDB deployments often store terabytes; scaling behavior may differ at larger scales and with different value distributions.
3) **One-at-a-Time Parameter Variation**: We vary parameters individually. Interaction effects between parameters (e.g., cache size interacting with memtable size) are not captured.
4) **Standardized Workloads**: db_bench and YCSB use synthetic key distributions and uniform request patterns. Real workloads exhibit skew, temporal locality, and time varying load that may interact with parameter choices differently.
5) **No Concurrent Background Load**: Experiments assume dedicated hardware with no competing workloads. Production environments must contend with OS processes and neighboring workloads.

Despite these limitations, the results provide valuable insights into parameter sensitivities and trade-offs that should qualitatively hold across environments.

IV Results: Compression Algorithms

A. Overview

Compression uses more CPU cycles during flush and compaction process for reduced disk I/O and a smaller database size. RocksDB can support multiple compression algorithms, each presenting a different balance between compression ratio and computational cost. We have evaluated four configurations: no compression as a baseline, Snappy, LZ4, and ZSTD.

B. Empirical Findings

TABLE I
COMPRESSION ALGORITHM COMPARISON (AVERAGE ACROSS DIFFERENT RECORD RUNS)

Algorithm	Throughput (ops/sec)	Avg DB Size (MB)	P99 Read Latency (µs)	Space Reduction
None	225,675	7,450	23.25	—
Snappy	264,441	4,462	19.75	40.1%
LZ4	269,799	4,462	19.50	40.2%
ZSTD	224,265	4,783	23.25	35.8%

The data support four main observations:

- **LZ4 achieves the best overall performance.** LZ4 achieved 269.8K operations per second while reducing database size by approximately 40%, outperforming every other algorithm we tested along. The small throughput improve over Snappy (about 1.9%) held steady across repeated runs, which we attribute to LZ4's faster decompression path.
- **Snappy matches LZ4 in compression ratio with slightly lower speed.** Snappy produced the same average database size (4,462 MB) but operated at 264.4K operations per second. Snappy retains value in deployments

that must support multiple programming languages, since its libraries are more widely available than LZ4's.

- **ZSTD carries a heavy CPU cost.** ZSTD reduced the database to 4,783 MB. It is a noticeably larger footprint than LZ4 or Snappy, corresponding to only a 35.8% space reduction while throughput fell to 224.3K operations per second, a 17% drop relative to LZ4. With 1 KB values, the CPU time spent on decompression is not recovered through space saving. ZSTD could become good for larger value sizes where its stronger compression ratios might justify the added processor load.
- **Running without compression is clearly not good option.** The uncompressed database consumed up to 7,450 MB, 67% larger than the LZ4 configuration. Rather than improving latency, the uncompressed setup produced a P99 read latency 23.25 µs and same to ZSTD and worse than both LZ4 and Snappy. While consuming extra disk space without any benefit.

C. Trade-off Analysis

Compression adds CPU overhead on every read and write, but it also compress the data that need to move across the storage bus and allows more records to place in the block cache. We measured a P99 read latency of 19.50 µs for LZ4, which is faster than the uncompressed baseline at 23.25 µs. This result appears counterintuitive until one recognizes that more compressed pages fit in cache, so few requests to reach the disk. Once the active working set grows above cache capacity, the decompression penalty finally shows and latencies increase. The same logic applies to ZSTD: at 1 KB values its computational demands are not offset by space savings, but as values grow to 4 KB or more, the reduction in disk traffic would likely cause the extra CPU cycles.

D. Practical Guidance

We recommend LZ4 as the default choice for almost all workloads because it provides the highest throughput with good space reduction for all the value sizes we studied. Snappy is a good alternative when the deployment involves several programming languages and library availability becomes a deciding factor. ZSTD should be reserved for large values like 4 KB and above, running on machines where disk space is limited and CPU is available. Disabling compression entirely offers no benefit: it inflates storage requirements and, in our tests, did not improve read latency compared with the faster compressed results.

V Results: Memtable Sizing

A. Overview

The memtable size is controlled by `write_buffer_size`, it determines how much data accumulate in memory before it flush to disk as an L0 SSTable. Larger memtables reduce flush frequency and allow more records to be sorted in memory, which can lower write amplification. The issue is increased RAM consumption and transient spikes in disk usage during flushes.

B. Empirical Findings

TABLE II
MEMTABLE SIZE IMPACT (AVERAGE ACROSS TWO DATASET SIZES)

Buffer Size	Throughput (ops/sec)	Avg DB Size (MB)	Flush Count	Flushes (total)
64 MB	257,095	4,410	672.0	highest
128 MB	253,464	5,218	336.0	2x fewer
256 MB	269,280	4,431	163.0	sweet spot
512 MB	288,242	9,376	84.0	transient spike

The data support four main observations:

- **Throughput rises with memtable size.** The 512 MB memtable reached 288.2K operations per second, 12.1% above the 64 MB configuration at 257.1K operations per second. This improvement shows lower flush overhead. The gain does not scale linearly, when doubling from 256 MB to 512 MB improves throughput by only 7.0%.
- **Flush frequency reduce by half with each doubling.** The 64 MB memtable resulted 672 flushes, while the 128 MB memtable pulled this to 336. The pattern suggests that flush overhead is bottleneck at small buffer sizes.
- **512 MB produces transient space amplification.** At 512 MB, the database grew up to 9,376 MB, roughly double the steady state size observed at 256 MB (4,431 MB). This occurs because the in-memory buffer itself occupies disk space in the flush operation. once the flush completes and compaction proceeds, the reduced back around 4,431 MB. In the systems with limited storage budgets, this oscillation can be problematic.
- **256 MB is a practical sweet spot.** It delivered 5.0% higher throughput than the 64 MB configuration also keeping the steady state database size nearly unchanged (4,431 MB compared to 4,410 MB) and reducing flush overhead to 163 flushes only.

C. Trade-off Analysis

Memtable sizing involves three important trade-off. First one is large memtables improve throughput by reducing flush count, yet each flush individually becomes more time taking because more data must be written and sorted. Second, on increasing memtable size from 64 MB to 256 MB does not significantly raise the steady state database size. The difference between 4,410 MB and 4,431 MB. At 512 MB, some significant spikes appear because the flush process itself consumes additional disk space for some time. Third, larger memtables improve average throughput at the cost of greater variance in individual write latencies, since large flushes can cause stalls. For usecase that require predictable latency, this variability may be unacceptable.

D. Practical Guidance

Set `write_buffer_size` to 256 MB as the default for most workloads, since this setting balances throughput improvement against transient space amplification. Consider 512 MB only when RAM is abundant and transient database size spikes of roughly 2x can be tolerated. This gives an additional 7% in throughput. On capacity constrained systems, use 128 MB to keep space oscillation under control to approximately 1.2x the steady state size while still halving flush frequency relative to 64 MB. Avoid very small buffers: the 64 MB configuration forced 672 flushes across an 8M record dataset, generating substantial background I/O that affect foreground throughput.

VI Results: Background Job Parallelism

A. Overview

The `max_background_jobs` parameter is for how many threads may run flushes and compactions concurrently in background. More threads allow compaction work to proceed in parallel, which can keep the LSM tree small and reduce write stalls. The risk is increased CPU contention and cache overhead.

B. Empirical Findings

TABLE III
BACKGROUND JOB PARALLELISM IMPACT

Max Threads	Throughput (ops/sec)	P99 Read Latency (µs)	Scaling Gain (%)
2	233,480	22.25	–
4	269,513	19.50	+15.4%
8	298,867	17.50	+11.0%
16	325,561	16.00	+8.9%

The data support four main observations:

- **Throughput scales significantly from 2 to 16 threads.** Each doubling produced meaningful gains: 15.4%, 11.0%, and 8.9% respectively, for a total improvement of 39.5% from 2 to 16 threads. This is noticeably better than typical foreground workload parallelism on the same hardware.
- **Read latency improves monotonically.** P99 read latency fell from 22.25 µs at 2 threads to 16.00 µs at 16 threads, a 28% reduction. The reason is that more compaction threads can keep the LSM tree small, so each read traverses very few levels.
- **Scaling remains sublinear.** The increment in results minimizes as threads increased: the step from 8 to 16 threads yielded only 8.9%, while 15.4% from 2 to 4 threads. This behavior reflects Amdahl's law: compaction is not perfectly parallelizable, and synchronization overhead also with resource contention become visible at higher thread counts.
- **The default of 4 threads is conservative.** On 16 core hardware, the default `max_background_jobs=4` leaves portion of parallelism unused. Operators with some extra CPU capacity should raise this parameter.

C. Trade-off Analysis

Increasing background thread counts shows clear benefits for throughput and read latency, but it also brings potential issues. Threads consume CPU cycles that might otherwise serve foreground requests. On already CPU bound systems, more background threads can degrade foreground latency despite improving compaction efficiency. Each thread also maintains local buffers, so excessive thread counts can pressure the page cache and memory bandwidth. Also more threads issuing I/O requests can overwhelm the operating system scheduler, especially on systems with limited I/O bandwidth. Our results were obtained on a network attached NTFS volume, so the gains may be smaller than what would be observed on local NVMe storage. The continued improvements even at 16 threads suggest that on multi core hardware the bottleneck is not thread management overhead but rather I/O and memory bandwidth.

D. Practical Guidance

On CPU-rich systems, set `max_background_jobs` to half the number of available cores: 8 on a 16 core machine, leaving 8 cores for foreground work, and 4 on 8 core machine, matching the current default. Adopt a measurement based approach: monitor CPU utilization and foreground latency while increasing background jobs, and stop when foreground process begins to suffer. On mobile or embedded systems, keep background threads minimal to preserve CPU for application logic. Also, note that Leveled compaction benefits more from parallelism than Universal compaction, so prioritize higher thread counts when using the Leveled.

VII Results: Block Cache Sizing

A. Overview

The block cache is an in-memory LRU cache that holds recently read data blocks. A large cache can reduces disk I/O for repeated reads, but memory is finite, and gives smaller once the cache exceeds the working set size.

B. Empirical Findings

TABLE IV
BLOCK CACHE SIZE IMPACT

Cache Size	Throughput (ops/sec)	P99 Read Latency (µs)	Marginal Gain (%)
512 MB	266,101	19.50	–
1 GB	300,534	17.25	+12.9%
2 GB	275,764	19.00	-8.2%
4 GB	284,279	18.25	+3.1%

The data support three main observations:

- **1 GB cache is the best.** The 1 GB configuration produced the highest throughput (300.5K operations per second) and the lowest P99 read latency (17.25 µs). The 512 MB cache suffered from cache misses that forced reads to disk, which raised latency.
- **Returns small and become noisy above 1 GB.** At 2 GB, throughput actually fell to 275.8K operations per second, suggesting increased cache management overhead or heap fragmentation. At 4 GB, throughput recovered to 284.3K operations per second but remained below the 1 GB figure. This pattern indicates that on each run variance affects at this scale, or that the working set is approximately 1 GB in size.
- **1 GB aligns with the working set.** With 8 to 10 million records at 1 KB values, the distinct data blocks accessed in the read heavy workload and consume roughly 1 GB. Once the cache exceeds this size limit, further increases cache no new data.

C. Trade-off Analysis

Block cache sizing is basically limited by the working set: the cache can only store what is actually accessed. Moving from 512 MB to 1 GB cost an additional 512 MB of memory and bought 12.9% higher throughput plus a 2.25 µs reduction in latency. Moving from 1 GB to 2 GB cost an additional 1 GB of memory and did not provide benefit clearly. Performance degraded slightly. These results are specific to the read random pattern on 8 to 10 million records that we tested. Applications with larger or more complex access patterns will have different working set sizes and therefore different optimal cache sizes. It is also worth noting that the RocksDB block cache interacts with L3 cache of CPU and the operating system page cache. A thorough analysis would require hardware performance profiling, which lies outside the scope of this study.

D. Practical Guidance

Characterize the working set size by running the actual workload and measuring cache hit rates through RocksDB statistics (`GetProperty("rocksdb.block-cache-usage")`). Size the block cache to 1.2 to 1.5 times the steady state working set to accommodate temporal shifts. Allocate 20% to 30% of available RAM to the block cache. On a 16 GB system this suggests 3 to 5 GB, though our results indicate diminishing returns beyond 1 GB per gigabyte of logical data. On memory constrained systems, prioritize block cache over memtable size, since a larger cache has a more direct impact on read latency. Above all, measure actual hit rates and adjust iteratively rather than relying on fixed rules.

VIII Results: Compaction Strategy

A. Overview

RocksDB offers four compaction strategies: Leveled, Tiered (Universal), FIFO, and K compaction technique. Each strategy makes different trade offs among write amplification (WA), read amplification (RA), and space amplification (SA). We evaluate three of these strategies. FIFO is included for completeness but yields unreliable results due to fast data deletion.

B. Empirical Findings

TABLE V
COMPACTION STRATEGY COMPARISON (8M RECORD DATASET, EXCLUDING FIFO)

Strategy	Throughput (ops/sec)	DB Size (MB)	P99 Read Latency (µs)	Space Ampl.
Level	443,515	860	9	**1.0x**
Universal	389,419	1,683	10	**1.96x**
FIFO	4,594,395	38	2	**–**

The data support two main observations:

- **Leveled compaction performs best on this workload.** Leveled compaction reached 443.5K operations per second, the highest throughput of any strategy tested, while keeping the database to 860 MB. This corresponds to 1.0x space amplification, meaning no redundant data remained after compaction. P99 read latency was 9 µs, the lowest among the comparable strategies.
- **Universal compaction trades space for write efficiency.** Universal compaction achieved 389.4K operations per second, roughly 12% below Leveled, but the database grew to 1,683 MB, nearly double the Leveled footprint. The larger size stems from Universal's less aggressive merging policy: it retains multiple overlapping files at each level rather than fully combining them. This reduces write amplification at the cost of higher read amplification and space amplification.

C. Trade-off Analysis

Choosing a compaction strategy is among the most consequential tuning decisions for an LSM tree. The strategies embody fundamentally different design goals. Leveled compaction enforces a strict hierarchy in which each level is a fixed multiple larger than the one below it (typically 10x). This structure demands more total merging, which raises write amplification, but it keeps the tree shallow, which lowers read amplification, and it removes obsolete data promptly, which keeps space amplification minimal. Leveled is therefore well suited to workloads with a mix of reads and writes. Universal compaction, by contrast, preserves overlapping files across generations at each level. This reduces the volume of merging, which lowers write amplification, but it deepens the effective tree, which raises read amplification, and it leaves redundant data in place, which raises space amplification. Universal is preferable when writes dominate and reads are infrequent. FIFO avoids merging entirely and simply drops the oldest files once a size limit is reached. This drives write amplification and space amplification toward zero, but it is applicable only to data that naturally expires, such as timeseries metrics.

D. Practical Guidance

For general purpose deployments, Leveled compaction is the safe default: it balances read and write performance and maintains excellent space efficiency. Reserve Universal compaction for workloads that are more than 90% writes with fewer than 10% reads, and only when disk space is plentiful; measure carefully before committing to it. Use FIFO exclusively for append timeseries data such as logs or metrics, and never for transactional workloads. If Leveled compaction is chosen, pay close attention to `max_bytes_for_level_base` and `max_bytes_for_level_multiplier`, since these parameters govern the tree geometry and strongly influence the trade-off between write and read amplification.

IX Discussion and Limitations

A. Key Takeaways

This empirical study reveals several important principles for RocksDB practitioners:

1) **Compression is helpful**: Use LZ4 by default. The space saving (40%) significantly outweigh any latency trade-offs, and LZ4 is generally faster than uncompressed due to reduced cache evictions.
2) **Memtable and parallelism are high ROI knobs**: Increasing memtable from 64 to 256 MB and background jobs from 2 to 8 provide large throughput improvement (15–20%) with minimal downside risk.
3) **Default is not always best**: Workload specific optimization is essential. Read heavy, mixed, and write heavy workloads require different strategies and parameters.
4) **Measurement is mandatory**: Static guidelines fail because workloads, storage, and hardware vary. Practitioners must measure actual performance and adjust iteratively.
5) **Storage matters more than initially apparent**: The drvfs I/O mode study revealed that the underlying storage layer (filesystem, device latency, bandwidth) has enormous impact on what configurations work well. Generalization across storage types is risky.

B. Study Limitations

We acknowledge several important limitations:

1) Environmental Constraints

1) **WSL2/drvfs is not representative of production**: NTFS filesystems, WSL2 guest-host communication overhead, and lack of true direct hardware access mean results may not directly transfer to Linux deployments on NVMe or SSD arrays.
2) **Limited scale**: 50M and 100M records are tiny compared to RocksDB deployments that manage terabytes. LSM tree behavior at larger scales may differ.Ccompaction costs may be different ratios, cache miss rates may follow different patterns.
3) **Fixed value size**: All experiments use 1 KB values. The effectiveness of compression algorithms, block cache, and compaction strategies changes dramatically with larger values (4-16 KB) or high variable sizes.

2) Experimental Methodology Limitations

1) **Single parameter variation**: We vary parameters individually to isolate effects, but RocksDB parameters interact in complex ways. The optimal configuration may not be a simple combination of individually optimal parameters.
2) **Synthetic workloads**: db_bench and YCSB use uniform key distributions and steady request rates. Real workloads have skewed distributions, temporal locality, time varying load, and bursty patterns that interact with parameters differently.
3) **No concurrent load**: Experiments assume dedicated hardware. Production systems compete with other workloads, OS processes, and variable system load, affecting parameter sensitivities.
4) **Limited run measurements**: Each configuration was measured minimum of 2 times. Multiple runs with variance analysis would improve confidence, though our results are generally consistent across repeated experiments.

3) Parameter Space Coverage

We evaluated a limited subset of RocksDB's tunable parameters. Important parameters not studied include:

- `level0_file_num_compaction_trigger`: Controls when L0 becomes too large
- `level0_slowdown_writes_trigger` and `level0_stop_writes_trigger`: Controls write stall behavior
- `target_file_size_base`: Controls individual SSTable file size
- `max_bytes_for_level_base` and `max_bytes_for_level_multiplier`: Control level size ratios
- Prefix bloom filter configuration

These parameters have significant impact and deserve dedicated study.

C. Implications for Future Work

Our study opens several directions for future research:

1) **Parameter interaction analysis**: Use factorial design of experiments (DoE) to systematically study how pairs or triples of parameters interact, moving beyond one-at-a-time variations.
2) **Workload-adaptive tuning**: Develop automated systems that monitor workload characteristics at runtime and adjust RocksDB parameters dynamically to maintain optimal performance.
3) **Machine learning for parameter selection**: Train models on large benchmarking datasets to predict optimal configurations given workload and hardware inputs.
4) **Large-scale study**: Repeat this study on terabyte-scale datasets on production storage (NVMe, SSD arrays) to validate that patterns hold at realistic scales.

5) **Real workload characterization**: Instrument production RocksDB deployments to understand actual workload patterns and validate that recommendations apply to real systems.

References

[1] Facebook/RocksDB, *RocksDB Wiki*, GitHub. [Online]. Available: https://github.com/facebook/rocksdb/wiki

[2] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Strum, "Optimizing Space Amplification in RocksDB," in *Proc. 8th Biennial Conf. on Innovative Data Systems Research (CIDR)*, Chaminade, CA, USA, Jan. 2017.

[3] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of Development Priorities in a Key-Value Store Serving Large-scale Applications," *ACM Trans. Storage*, vol. 17, no. 4, Article 26, pp. 1–32, Oct. 2021.

[4] C. Luo and M. J. Carey, "LSM-based Storage Techniques: A Survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, Jan. 2020.

[5] S. Sarkar and M. Athanassoulis, "Dissecting, Designing, and Optimizing LSM-based Data Stores," in *Proc. ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '22)*, Philadelphia, PA, USA, Jun. 2022, pp. 2489–2497. https://doi.org/10.1145/3514221.3522563

[6] P. O'Neil, E. Cheng, D. Gawlick, and E. J. O'Neil, "The Log-Structured Merge-Tree (LSM Tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[7] Y. Matsunobu, S. Dong, and H. Lee, "MyRocks: LSM-Based Storage Engine with Zoned Namespace SSD Optimization," in *Proc. 21st Usenix Conf. on File and Storage Technologies (FAST '23)*, Santa Clara, CA, USA, Feb. 2023.

[8] Y. Q. Liu, Y. M. Guo, and C. Zheng, "TiKV: A Distributed, Transactional Key-Value Database," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3582–3595, 2020.

[9] S. Dutta, S. Hellerstein, A. Phanishayee, and S. Setty, "CockroachDB: The Resilient Geo-Distributed SQL Database," in *Proc. ACM SIGMOD Int. Conf. on Management of Data (SIGMOD '20)*, Portland, OR, USA, Jun. 2020, pp. 1645–1649.